Submission 2 — Technical Report

FinCommerce Assistant (Qdrant + CLIP)

Team: 4QUBITS

January 26, 2026

GitHub Repo: https://github.com/Belaiiiid/4QUBITS.git
Qdrant Collection: `vio_products_clip` (Vectors: `text`, `image`)

1 Idea & Problem Statement

Classic e-commerce search and recommendations often ignore a user's financial constraints (budget/affordability) and do not fully leverage multimodal understanding (text + images). Our project builds a **non-chatbot** assistant for product discovery and recommendations, using similarity search to retrieve relevant items and provide explainable, constraint-aware results.

Hackathon alignment. The solution is not a conversational chatbot; it is a retrieval and recommendation application centered on vector search with Qdrant as the main vector database, producing actionable ranked outputs with explainability and an evaluation plan.

2 Solution Overview

We provide an "Assistant" web application (FastAPI backend + Next/HTML frontend) where the user enters:

- a natural-language intent (e.g., "wireless noise cancelling headphones for travel"),
- a budget,
- an optional category constraint,
- top-k results to display.

The UI returns:

- **Within-budget recommendations** (strict budget compliance),
- **Near-budget recommendations** (slightly above budget, e.g., up to +10–15%, but strong match),
- **Alternatives** (close substitutes within budget if the best matches are expensive).

3 Architecture (Including Qdrant Integration)

3.1 High-level Components

- **Frontend (React/HTML):** form input (query, budget, category), results view, explainability panel, events logging calls.

- **Backend (FastAPI):** query embedding (CLIP text), Qdrant querying + filtering, reranking, explanation generation, event logging, metrics endpoint.
- **Vector DB (Qdrant Cloud):** stores product embeddings and payload; performs similarity search and payload filtering.

3.2 Data Flow

1. User submits a query + budget (+ optional category).
2. Backend embeds the query using CLIP (text encoder) to obtain a vector representation.
3. Backend queries Qdrant using the named vector `text` and applies payload filters (e.g., `price <= budget`, optional `category`).
4. Backend returns top-k items (in-budget), plus near-budget items (slightly above budget) and alternatives when needed.
5. Frontend displays results and sends interaction events (impression/click/add_to_cart) to backend.
6. Backend stores events for metrics and feedback-loop improvements.

3.3 Qdrant Schema

We store one point per product with:

- **Named vectors:** `text` and `image` (CLIP embeddings).
- **Payload fields:** `product_id`, `name`, `category`, `subcategory`, `price`, `price_bucket`, `rating`, `total_ratings`, `bayes_rating`, `value_score`, `image_url`.

Filtering. We create payload indexes for fields used in filtering (e.g., `price` and `category`) to support fast constraint-aware retrieval.

4 Data Pipeline

4.1 Initial Data (Catalog)

Our starting dataset is a product catalog containing product metadata and a reference to product images. Core fields include:

- identifiers: `Product ID`,
- taxonomy: `Product Category`, `Product Subcategory`, `price_bucket`,
- pricing: `Product Price ($)`,
- quality signals: `Product Rating`, `Total Ratings`, `Product Bayes_rating`, `Product Value_score`,
- media: `Product Image` (URL) and local images on Drive named by `Product ID`.

4.2 Preprocessing & Feature Engineering

- Normalize types (price as float, ratings as float/int).
- Build a text document per product from name + category + subcategory + price bucket for embedding.
- Ensure stable Qdrant IDs (UUID-based) to avoid invalid IDs (e.g., 0) and support re-ingestion.

4.3 Embeddings & Ingestion

- Compute CLIP **text** embeddings from the constructed product text.
- Compute CLIP **image** embeddings from local images (when available).
- Upload points in chunks/batches to Qdrant Cloud (robust ingestion for tens of thousands of products).

4.4 Runtime Data (User Behavior)

During usage, we generate new "future" data:

- **Events:** impressions, clicks, add-to-cart actions.
- **Stored fields:** session_id, timestamp, query, budget, product_id, rank, list_type (in-budget/near-budget/alternative).

This behavior data supports analytics (CTR, add-to-cart rate) and a feedback loop to refine ranking.

5 Ranking & Recommendation Policy

5.1 Constraint-aware Retrieval

Primary results are retrieved with strict constraints:

- `price <= budget`
- optional category filter (e.g., `category == electronics`)

5.2 Near-budget Recommendations

We optionally allow near-budget items:

- `budget < price <= budget * 1.15`
- only shown if similarity score is strong (to justify the budget stretch)

5.3 Alternatives Generation

If in-budget matches are scarce:

- relax category constraint while keeping budget, or
- retrieve similar substitutes (closest semantic neighbors) within budget.

6 Explainability

Each recommendation includes short evidence-based explanations, for example:

- semantic match to the user's intent (vector similarity),
- budget compliance or small budget stretch (+%),
- quality/value indicators (Bayes rating, value score),
- category/subcategory alignment.

7 Evaluation Plan

7.1 Offline Evaluation

- Create a small query set (10–20 realistic intents).
- Measure Precision@K or manual relevance checks (top-k judged relevant/not relevant).
- Track constraint compliance rate (fraction of in-budget results satisfying budget).

7.2 Online/Interactive Metrics

From event logs:

- CTR = clicks / impressions
- Add-to-cart rate = add_to_cart / impressions
- Out-of-budget recommendation rate (near-budget share)

8 Project Timeline

8.1 Work Completed

- Data preprocessing and feature preparation (price, rating, value signals).
- CLIP embedding pipeline (text + images) and Qdrant Cloud integration.
- Initial ingestion (chunked upload) and query notebook for retrieval + filtering.

8.2 Next Steps

- Finalize FastAPI endpoints: search, events logging, metrics.
- Build frontend UI with sections: in-budget, near-budget, alternatives.
- Implement feedback loop reranking using session preferences from clicks.
- Improve evaluation with a fixed test set and documented results.